



Homework 2

Please complete the following problems. You must submit the following for each problem: a) the answer you are asked to return and b) Python code that can be run to verify your work.

1. Introduction to the Bioinformatics Armory

If you are an accomplished coder, then you can write a separate program for every new task you encounter. In practice, these programs only need to be written once and posted to the web, where those of us who are not great coders can use them quickly and efficiently. It is valuable to familiarize yourself with some of the more popular bioinformatics tools taken from "out of the box" software.

To be equitable, the focus will be mainly on free, internet-based software and on programs that are compatible with multiple operating systems. The "Problem" section will contain links to this software, with short descriptions about how to use it.

PROBLEM

To solve this initial problem, you merely must take a given DNA sequence and find its nucleotide counts.

Of the many tools for DNA sequence analysis, one of the most popular is the [Sequence Manipulation Suite](#). Commonly known as SMS 2, it comprises a collection of programs for generating, formatting, and analyzing short strands of DNA and polypeptides.

One of the simplest SMS 2 programs, called **DNA stats**, counts the number of occurrences of each nucleotide in a given strand of DNA. An online interface for **DNA stats** can be found [here](#).

Given: A file containing a DNA string s of length at most 1000 bp.

Return: Four integers (separated by spaces) representing the respective number of times that the symbols 'A', 'C', 'G', and 'T' occur in s . Note: You must provide your answer in the format shown in the sample output below.

Note: instead of code, please submit a screenshot of your SMS 2 DNA stats output.

Sample Dataset

```
AGCTTTTCATTCTGACTGCAACGGGCAATATGTCTCTGTGTGGATTAAAAAAGAGTGTCTGATAGCAGC
```

Sample Output

```
20 12 17 21
```



Homework

Bioinformatics Engineering

ENBC455

Hint: Although the problem is asking you to use an online tool, you can easily use the built-in Python method `.count()` for strings. Here's how you could count the occurrences of 'A' found in a Biopython **Seq** object.

```
>>> from Bio.Seq import Seq
>>> my_seq = Seq("AGTACACTGGT")
>>> my_seq.count("A")
3
```



2. GenBank Introduction

The most comprehensive database available for molecular biologists is GenBank, an open access resource that contains an annotated collection of all publicly available sequenced DNA and its translation into proteins. GenBank was founded by NCBI in 1982, and over the last four decades, the data it houses has grown exponentially. As of 2023, GenBank had about 2.5 trillion nucleobases. GenBank is constantly updated by direct submissions from individual laboratories, as well as bulk submissions from large-scale sequencing centers.

Every sequence has a unique GenBank identifier that will directly retrieve its full sequence record. Here are some examples of database IDs:

CAA79696
NP_778203
263191547
BC043443
NM_002020

You can also search by submitter/author name in the format last_name first_initials (e.g., Smith JR).

To search for an exact match, enclose it in quotation marks.

"contactin associated protein"
"duchenne muscular dystrophy"

You can also restrict your search or use specific subsets of records by using Boolean operators (AND, OR, NOT).

GenBank uses its file format for storing genome data. Check out a full GenBank entry [here](#).

GenBank comprises several subdivisions: [Nucleotide](#) is a collection of nucleic acid sequences from several sources. Searching the Nucleotide database with general text queries will produce the most relevant results. You can also use a simple query based on protein name, gene name, or gene symbol.

A gene symbol is a short-form abbreviation for a particular gene. Gene nomenclature rules were established by the Nomenclature Committee of the Human Genome Organization (HUGO) and were later adopted by other species-specific research communities. Although HUGO assigns every known human gene its own "long" name, they also assign the gene a "short" symbol.

In general, HUGO requires that gene symbols:

- are unique
- contain only Latin letters and Arabic numerals (0, 1, 2, ...)
- do not contain punctuation
- do not contain any reference to the species they are encoded in (i.e., "H/h" for humans)

For two examples of gene symbols, the gene "alcohol dehydrogenase" in *Drosophila melanogaster* has the symbol Adh, and DNA polymerase gamma in *Homo sapiens* has the symbol POLG.



To limit your search to only certain kinds of records, you can use the filters on the left-side of the search results screen. If you cannot find what you are searching for, check how the database interpreted your query by investigating the **Search details** field on the right side of the page. This field automatically translates your search into standard keywords.

For example, if you search for **Drosophila**, the **Search details** field will contain (**Drosophila[All Fields]**), and you will obtain all entries that mention *Drosophila* (including all its endosymbionts, an organism that lives inside another organism). You can restrict your search to only organisms belonging to the *Drosophila* genus by using a search tag and searching for **Drosophila[Organism]**.

The genus is a low-level rank used in biological taxonomy. In the hierarchy of major taxonomic ranks, genus falls one step above species and one step below family. The genus is used in an organism's "Linnaean name", which names an organism by its genus and species (e.g., humans belong to the genus and species *Homo sapiens*).

PROBLEM

Given: A file containing a genus name, followed by two dates in YYYY/M/D format.

Return: The number of Nucleotide GenBank entries for the given genus that were published between the dates.

Note: instead of code, please submit a screenshot of the GenBank page showing the correct result.

Sample Dataset

Anthoxanthum
2003/7/25
2005/12/27

Sample Output

7

Hint: NCBI's databases, such as PubMed, GenBank, GEO, and many others, can be accessed via Entrez, a data retrieval system offered by NCBI. Entrez is a search engine that can search across all NCBI databases at the same time. For direct access to Entrez, you can use Biopython's **Bio.Entrez** module.

The **Bio.Entrez.esearch()** function will search any of the NCBI databases. This function takes the following arguments:

- **db**: The database to search. For example, this field can be **nucleotide** for GenBank or **pubmed** for PubMed.
- **term**: The search term for the "Query" field. You can use search tags here.



The below codes performs a quick search for the *rbcl* gene in corn (*Zea mays*):

```
>>>from Bio import Entrez
>>>Entrez.email = "your_name@your_mail_server.com"
>>>handle = Entrez.esearch(db="nucleotide", term='"Zea mays"[Organism] AND rbcl[Gene]')
>>>record = Entrez.read(handle)
>>>record["Count"]
'6' # Surely this value will change over time because GenBank is constantly updated
```

Note that when you request Entrez databases you must obey [NCBI's requirements](#):

- For any series of more than 100 requests, access the database on the weekend or outside US peak times.
- Make no more than three requests every second.
- Fill in the Entrez.email field so that NCBI can contact you if there is a problem.
- Be sensible with your usage levels; if you want to download whole mammalian genomes, use NCBI's [FTP](#).



3. Data Formats

Several different data formats have been used to represent genetic strings. The history of file formats presents its own kind of evolution: some formats have died out, being replaced by more successful ones.

The FASTA format (.fasta, .fas, .fa) is used by practically all modern software and databases. In FASTA files, a string is introduced by a line that begins with '>', followed by some information labeling the string. Subsequent lines contain the string itself; the next line beginning with '>' indicates that the current string is complete and begins the label of the next string in the file.

GenBank hosts its own file format for storing genome data, containing a large amount of information about each interval of DNA. The GenBank file describes the interval's source, taxonomic position, authors, and features (see Figure 1).

```
LOCUS SCU49845 5028 bp DNA PLN 21-JUN-1999
DEFINITION Saccharomyces cerevisiae TCP1-beta gene, partial cds, and Axl2p
              (AXL2) and Rev7p (REV7) genes, complete cds.
ACCESSION U49845
VERSION U49845.1 GI:1293613
KEYWORDS .
SOURCE Saccharomyces cerevisiae (baker's yeast)
  ORGANISM Saccharomyces cerevisiae
            Eukaryota; Fungi; Ascomycota; Saccharomycotina; Saccharomycetes;
            Saccharomycetales; Saccharomycetaceae; Saccharomyces.
REFERENCE 1 (bases 1 to 5028)
AUTHORS Torpey,L.E., Gibbs,P.E., Nelson,J. and Lawrence,C.W.
TITLE Cloning and sequence of REV7, a gene whose function is required for
        DNA damage-induced mutagenesis in Saccharomyces cerevisiae
JOURNAL Yeast 10 (11), 1503-1509 (1994)
PUBMED 7871890
REFERENCE 2 (bases 1 to 5028)
AUTHORS Roemer,T., Madden,K., Chang,J. and Snyder,M.
TITLE Selection of axial growth sites in yeast requires Axl2p, a novel
        plasma membrane glycoprotein
JOURNAL Genes Dev. 10 (7), 777-793 (1996)
PUBMED 8846915
REFERENCE 3 (bases 1 to 5028)
AUTHORS Roemer,T.
TITLE Direct Submission
JOURNAL Submitted (22-FEB-1996) Terry Roemer, Biology, Yale University, New
        Haven, CT, USA
```

Figure 1: An example of a GenBank record.

A sample GenBank entry can be found [here](#). You may export an entry to a variety of file formats by selecting the appropriate file format under the **Send To:** dropdown menu at the top of the page.

In Python, we can again use the Biopython **Bio.Entrez** module. To search for particular access IDs, you can use the function **Bio.Entrez.efetch(db, rettype)**, which takes two parameters: the **db** parameter takes the database to search, and the **rettype** parameter takes the data format to be returned. For example, we use "nucleotide" (or "nuccore") as the **db** parameter for Genbank and "fasta" as the **rettype** parameter for FASTA format.

The following code illustrates **efetch()** in action. It obtains plain text records in FASTA format from NCBI's [Nucleotide] database.

```
>>>from Bio import Entrez
>>>Entrez.email = "your_name@your_mail_server.com"
>>>handle = Entrez.efetch(db="nucleotide", id=["FJ817486", JX069768, JX469983"],
rettype="fasta")
>>>records = handle.read()
>>>print(records)
```



To work with FASTA format, we can use the **Bio.SeqIO** module, which provides an interface to input and output methods for different file formats. One of its main functions is **Bio.SeqIO.parse()**, which takes a handle and format name as parameters and returns entries as SeqRecords.

```
>>>from Bio import Entrez
>>>from Bio import SeqIO
>>>Entrez.email = "your_name@your_mail_server.com"
>>>handle = Entrez.efetch(db="nucleotide", id=["FJ817486", "JX069768", "JX469983"],
rettype="fasta")
>>>records = list(SeqIO.parse(handle, "fasta")) #we get the list of SeqIO objects in FASTA
format
>>>print(records[0].id) #first record id
gi|227437129|gb|FJ817486.1|
>>>print(len(records[-1].seq)) #length of the last record
771
```

PROBLEM

GenBank can be accessed [here](#). A detailed description of the GenBank format can be found [here](#).

Given: A file containing a collection of n ($n \leq 10$) GenBank entry IDs.

Return: The shortest of the strings associated with the IDs in FASTA format.

Note: please write a Python script to solve the problem. You may navigate the GenBank website to check your answer if you wish.

Sample Dataset

FJ817486 JX069768 JX469983

Sample Output

```
>JX469983.1 Zea mays subsp. mays clone UT3343 G2-like transcription factor mRNA, partial cds
ATGATGTATCATGCGAAGAATTTTTCTGTGCCCTTTGCTCCGCAGAGGGCACAGGATAATGAGCATGCAA
GTAATATTGGAGGTATTGGTGGACCCAACATAAGCAACCCTGCTAATCCTGTAGGAAGTGGGAAACAACG
GCTACGGTGGACATCGGATCTTCATAATCGCTTTGTGGATGCCATCGCCCAGCTTGGTGGACCAGACAGA
GCTACACCTAAAGGGGTCTCACTGTGATGGGTGTACCAGGGATCACAATTTATCATGTGAAGAGCCATC
TGCAGAAGTATCGCCTTGCAAAGTATATACCCGACTCTCTGCTGAAGGTTCCAAGGACGAAAAGAAAGA
TTCGAGTGATTCCCTCTCGAACACGGATTCCGGCACCAGGATTGCAAATCAATGAGGCACTAAAGATGCAA
ATGGAGGTTTCAAGCGACTACATGAGCAACTCGAGGTTCAAAGACAACCTGCAACTAAGAATTGAAGCAC
AAGGAAGATACTTGACAGATGATCATTGAGGAGCAACAAAAGCTTGGTGGATCAATTAAGGCTTCTGAGGA
TCAGAAGCTTTCTGATTCACCTCCAAGCTTAGATGACTACCCAGAGAGCATGCAACCTTCTCCCAAGAAA
CCAAGGATAGACGCATTATCACCAGATTGAGAGCGGATACAACACAACCTGAATTCGAATCCCATTGTA
TCGGTCCGTGGGATCACGGCATTGCATTCCCAGTGGAGGAGTTCAAAGCAGGCCCTGCTATGAGCAAGTC
A
```

4. Finding a Motif in DNA

A motif is an interval of nucleotides (in nucleic acids) or of amino acids (in proteins) that has biological importance. The context of this importance can vary; a motif could represent an important functional unit of a protein shared by many members of the same species, or it could represent a rare gene encoding a disorder. In all cases, a motif will be represented by a substring of a genetic string that we wish to locate.

Finding the same interval of DNA in the genomes of two different organisms (often taken from different species) is highly suggestive that the interval has the same function in both organisms. A common task in molecular biology is to search an organism's genome for such motif.

The situation is complicated by the fact that genomes are riddled with intervals of DNA that occur multiple times (possibly with slight modifications), called repeats. These repeats occur far more often than would be dictated by random chance, indicating that genomes are anything but random and in fact illustrate that the language of DNA must be very powerful (compare with the frequent reuse of common words in any human language).

The most common repeat in humans is the Alu repeat, which is approximately 300 bp long and recurs around a million times throughout every human genome (see Figure 1). However, Alu has not been found to serve a positive purpose and appears in fact to be parasitic: when a new Alu repeat is inserted into a genome, it frequently causes genetic disorders.

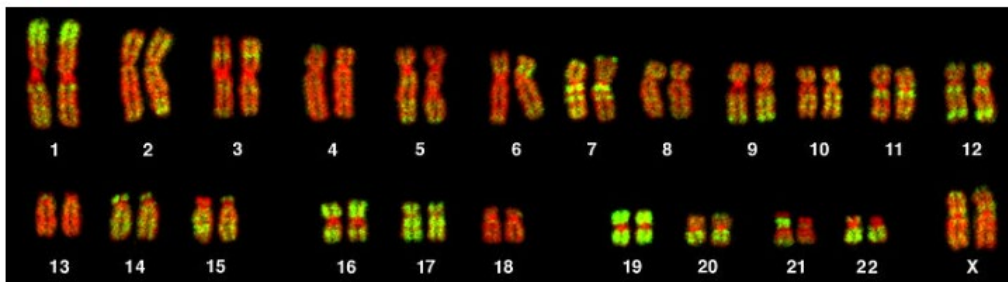


Figure 1: The human chromosomes stained with a probe for Alu elements, shown in green.

PROBLEM

For two strings s and t , t is a substring of s if t is contained as a contiguous collection of symbols in s (as a result, t must be no longer than s).

The position of a symbol in a string is the total number of symbols found to its left, including itself (e.g., the positions of all occurrences of 'U' in "AUGCUUCAGAAAGGUCUUACG" are 2, 5, 6, 15, 17, and 18). The symbol at position i of s is denoted by $s[i]$.

A substring of s can be represented as $s[j:k]$, where j and k represent the starting and ending positions of the substring in s ; for example, if $s = \text{"AUGCUUCAGAAAGGUCUUACG"}$, then $s[2:5] = \text{"UGCU"}$.

The location of a substring $s[j:k]$ is its beginning position j ; note that t will have multiple locations in s if it occurs more than once as a substring of s (see the Sample below).



Given: A file containing two DNA strings s and t (each of length at most 1 kb).

Return: All locations of t as a substring of s .

Sample Dataset

GATATATGCATATACTT

ATAT

Sample Output

2 4 10

Hint: Different programming languages use different notations for positions of symbols in strings. Above, 1-based numbering is used, as opposed to 0-based numbering, which is used in Python.

For $s = \text{"AUGCUUCAGAAAGGUCUUACG"}$, 1-based numbering would state that $s[1] = \text{'A'}$ is the first symbol of the string, whereas this symbol is represented by $s[0]$ in 0-based numbering. The idea of 0-based numbering propagates to substring indexing, so that $s[2:5]$ becomes "GCUU" instead of "UGCU".

Note that in some programming languages, such as Python, $s[j:k]$ returns only fragment from index j up to but not including index k , so that $s[2:5]$ becomes "UGC", not "UGCU".